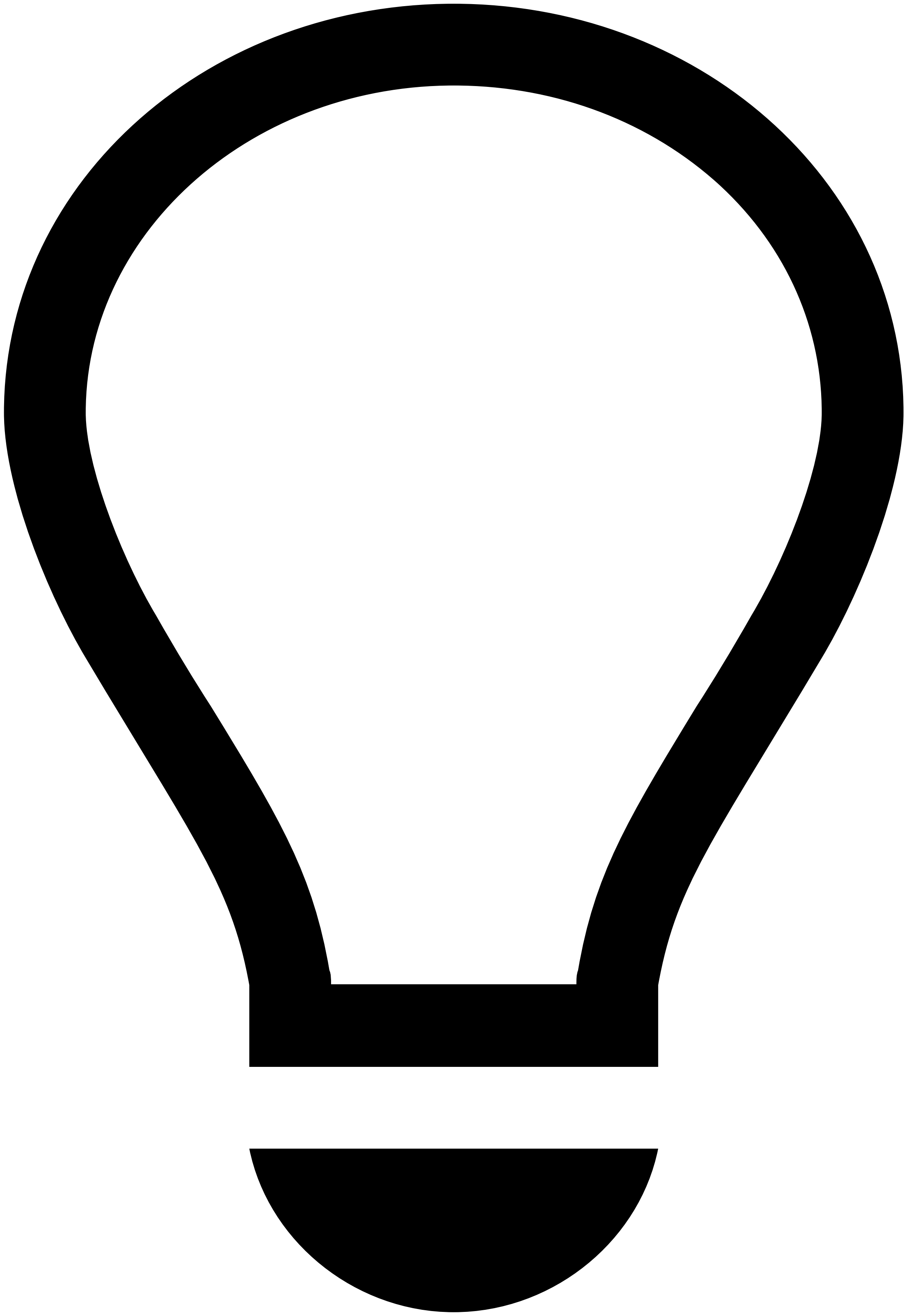
AutoML

Was this page helpful?Â

AutoML automates feature engineering, model training and hyperparameter tuning. The following topic describes in more detail what AutoML does.

Why use AutoML?â

Finding the right features and models for a data set is a cumbersome task: creating features, training models, tuning their parameters and testing the changes are all repetitive tasks. AutoML automates these tasks, using the fraud detection experience of Feedzai. Data Scientists can kick-off new fraud fighting projects with AutoML, or can use it to find interesting features for existing models.



Feedzai IQ

Further time savings can be achieved with [Feedzai IQ](), which provides ready-to-adopt risk assessment solutions that don't require extensive data science work, thanks to federated financial risk intelligence aggregated from the Feedzai global network.

Prerequisitesâ

To be able to run AutoML against a data source, you need to prepare the data first.

- Date and datetime fields need to be converted to Unix timestamp format.
- The data cannot contain NULL values. Instead, you need to add a default value:
    - Strings coalesced to empty string ('').
    - Categoricals should be coalesced to an existing value or to a new categorical value representing the NULL, typically a value called "other".
    - Numericals should be coalesced to a meaningful number, for example -999, -1 or 0, depending on their valid range.
    - Dates and datetimes should be coalesced to a default value, for example-2208988800, which corresponds to 01-01-1900 in Unix timestamp format.
- The input data source can't use empty string ('') as NULL.

Feature engineeringâ

Feature engineering is the process of applying domain knowledge to derive new characteristics from existing data. These new characteristics, the **features** might help to build better machine learning models for a specific purpose.

In Pulse, features are represented by Maps and Metrics in a Plan. When executing a Plan, Pulse processes the input data and creates a new data source that contains new fields. The value of those fields is defined by the Map and Metric operators of the Plan.

AutoML has a list of data field transformations that can be useful according to Feedzai's experience in the fraud fighting domain. Those transformations are based on the semantics of the data fields, combining fields with specific meaning into features. For example, if a data source has two location data fields, it might be useful to calculate the distance between the two. So, AutoML has a transformation that computes the distance between two location fields.

To allow AutoML to apply those transformations, when defining an AutoML run you need to provide a semantic file that maps the input data fields to semantic tags. For example, the *Name* field can be tagged with the *name* tag. This way AutoML knows the meaning of this input field, and it can compute the number of words in the name, which can be useful to fight fraud.

In summary, AutoML follows these steps to create features:

1. Looks at the semantic file and maps the data fields to their meaning.
2. Based on the data fields' meaning and on the list of transformations, AutoML creates a plan containing Map and Metric operators.
3. Executes the plan on the input data source, and saves the result in a **features data source** that contains the data enriched with the new data fields.

Dataset splitâ

When you create the AutoML run, you define which parts of the input data source will be used for training and for validation. Optionally you can also choose to have a test data set to simulate a blind evaluation of the best model found.

You define these data sets by selecting the time range for each task: training, validation, and optionally test. That means for example that data records that have a timestamp falling into the training time range will be used for training the models.

AutoML creates the training, validation and optionally the test data source based on the defined time ranges.

Planning training, validation and test data sources

To avoid under- or overfitting, plan the time ranges carefully. Ensure that the resulting data sets contain enough data for their purpose. A typical ratio is 50% for the training, 30% for the validation, and 20% for the evaluation data set.

Note that if you don't define a test data set, AutoML uses the validation data set for the final evaluation of the best model found. Since the validation data set was used for finding the best model, it is not an optimal solution to use it to test the data. Hence it's recommended to use a test data set.

Filtersâ

If the input data source contains different types of events, and you don't want AutoML to consider all of them when creating metrics or training models, you can apply filters.

AutoML allows you to define the definition of two kinds of filters:

- If you define a **profile filter**, AutoML will create metrics that consider events filtered by the defined expression. Example: you might want to filter for only transaction submissions and payment updates - `event_type = "transaction_submission" OR event_type = "transaction_payment_status_update"`
- If you define a **scorable filter**, AutoML will train the models based on events that satisfy the expression of this filter. Example: you might want to filter for only transaction submissions - `event_type = "transaction_submission"`

Filters in the first step of the AutoML run

The scorable filter will create a new filter operator in the plan that splits the data source.

Datasource Splitting plan with a Scorable Filter

Feature selectionâ

In this step, AutoML first trains an H2OGradient Boosted Trees model on a sample of the training dataset. GBT models are quick to be trained, and provide a feature importance ranking out of the box.

Then Pulse reduces the features to the subset that has the highest feature importance according to the GBT model. This way, Pulse keeps only a subset of the features for the model training step of AutoML. The features that AutoML keeps contribute with 90% of the importance rating.

Note that Pulse has another feature importance calculation method as well, which you can use later on, after you evaluated a trained model. That feature importance calculation method is different from the process that AutoML uses.

Reduced features planâ

The AutoML Pipeline combines the plan generated in the **Feature Engineering** step with the subset of features from the **Feature Selection** step, and reduces the plan. It does so by removing the least important features discovered in the Feature Selection step. This prevents data scientists from manually removing the operators that will produce useless features.

Model training and evaluationâ

After selecting the most important features, AutoML creates a Hyperparameter Tuning job to train and evaluate models. Hyperparameter Tuning jobs distribute the model training and evaluation in the Spark cluster.

Finally, AutoML compares the models based on the metric that you selected to be optimized, and recommends the best model.

Learn moreâ

- Creating an AutoML run
- Plans
- Maps and Metrics
- Semantic file
- Hyperparameter Tuning